

Greedy Algorithms.

Consider infinite supply of the following value coins.

10	5	2	1
----	---	---	---

If someone asks for an amount, how will you give this amount using minimum coins?

Amount : 52

Take 5 coins of 10Rs.

Take 1 coin of 2Rs.

Amount : 57

Take 5 coins of 10Rs.

Take 1 coin of 5Rs.

Take 1 coin of 2Rs.

```
int minCoins(int coins[], int n, int amount)
```

```
{  
    ① sort coins[] in decreasing order.
```

```
    ② int res = 0;
```

```
    ③ for (int i=0; i<n; i++)
```

```
    {
```

```
        if (coins[i] <= amount)
```

```
        {  
            int c = floor(amount / coins[i]);
```

```
            res = res + c;
```

```
            amount = amount - c * coins[i];
```

```
            if (amount == 0)
```

```
                break;
```

```
        }
```

```
    }  
    return res;
```

amount = 57

coins[] = [5, 10, 2, 1]

① After Sorting :

coins[] = [10, 5, 2, 1]

② res = 0

③ i=0 : res=5, amount=7

i=1 : res=6, amount=2

i=2 : res=7, amount=0

General Structure:

getOptimal (Item arr[], int n)

① Initialise: res = 0

② while (All items are not considered)

 i = selectAnItem()

 if (feasible(i))

 res = res + i

③ Return res

★ Greedy Algorithm may not work Always

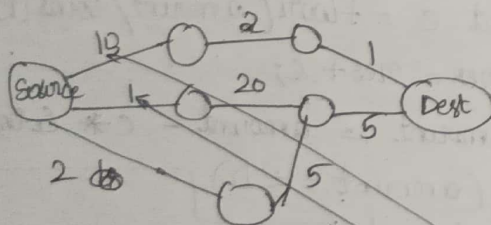
Consider coins[] = {18, 1, 10}

amount[] = 20

greedy
20 - 18 = 2
2 - 1 = 1 } 3 coins

but $\Rightarrow$ 20 - 10 = 10
10 - 10 = 0 } 2 coins

Another Example problem: Longest path



for longest path we choose 10 first, but there exist a much longer path with 1

Applications:-

→ finding optimal solutions

• Activity Selection

• Fractional knapsack

• Job sequencing

• Prim's Algorithm

• Kruskal's Algorithm

• Dijkstra's Algorithm

• Huffman Coding $\Rightarrow$ An optimal lossless compression technique

→ finding close to optimal solutions

for NP Hard problems like

Travelling salesman problem

Activity Selection Problem

Ip: $[(2,3), (1,4), (5,8), (6,10)]$

Op: 2

maximum no of activities that can happen on a single tasking Machine

Ip: $[(1,3), (2,4), (3,8), (10,11)]$

Op: 3

Naive: Consider every activity as the 1st activity then recursively find out maximum activities that we can do with this activities. take the max of all activities. $\Rightarrow O(\text{exponential})$

Ip: $[(3,8), (2,4), (1,3), (10,11)]$

① Sort according to finish time: $\Rightarrow n \log n$

$[(1,3), (2,4), (3,8), (10,11)]$

② Initialize solution as first activity

③ Do following for remaining activities

- If current activity overloads with the last picked activity in the solution ignore the current activity.
- Else add the current activity to the solution.

$[(1,3), (2,4), (3,8), (10,11)]$

$S = [(1,3)]$

$S = [(1,3), (3,8)]$

$S = [(1,3), (3,8), (10,11)]$

$S_1 = [(x_1, y_1), (x_2, y_2), \dots] \Rightarrow n_1$

$S_2 = [(y_1, y_2), (y_3, y_4), \dots] \Rightarrow n_2$

$n_2 > n_1$

int maxActivity(pair<int, int> arr[], int n)

```
{
    sort(arr, arr+n, mycmp);
    int prev=0;
    int res=1;
    for (int curr=1; curr<n; curr++)
    {
        if (arr[curr].first >= arr[prev].second)
        {
            res++;
            prev=curr;
        }
    }
    return res;
}
```

for (int curr=1; curr<n; curr++)

{ if (arr[curr].first >= arr[prev].second)

{ res++;

prev=curr;

}

return res;

}

arr[] = $[(12, 25), (10, 20), (20, 30)]$

After sorting:

$[(10, 20), (12, 25), (20, 30)]$

res=1, prev=0

curr: 1, Ignore

curr: 2, res=2, prev=2

Qp: Fractional knapsack

	I_1	I_2	I_3
weights	50	20	30
Values	600	500	400

Knapsack capacity = 70

Qp: 1140

$$\begin{aligned}
 & 20 + 30 \\
 & I_2 + I_3 + I_1 \times \frac{20}{50} \\
 & 500 + 400 + \frac{12}{50} \times 600 \\
 & 500 + 400 + 240 \\
 & = 1140
 \end{aligned}$$

Qp:

	I_1	I_2	I_3
weights	10	15	20
Values	200	50	100

Knapsack capacity = 15

Qp: 250

$I_1 + I_2$

- Calculate ratio (value/weight) for every item.
- Sort all item in decreasing order of the ratio.
- Initialise: $res = 0$, $curr_cap = \text{given_cap}$;
- Do following for every Item I in Sorted order
 - Else if ($I.\text{weight} \leq \text{curr_cap}$)
 - $\text{curr_cap} -= I.\text{weight}$;
 - $res += I.\text{Value}$;
 - Else
 - $res += (I.\text{value}) * (\text{curr_cap} / I.\text{weight})$;
 - return res ;
- Return res

Knapsack capacity = 70

	I_1	I_2	I_3
weights	50	20	30
values	600	500	400

Value/weight $12 \quad 25 \quad 40/3$

After Sorting by Value/weight

	I_2	I_3	I_1
weight	20	30	50
Value	500	400	600

$curr_cap = 70$, $res = 0$

pick I_2 : $curr_cap = 50$, $res = 500$

pick I_3 : $curr_cap = 20$, $res = 900$

partially pick I_1 : $curr_cap = 0$, $res = 1140$

$$900 + \frac{20}{50} \times 600$$

$$900 + 240$$

double $frknaps(\text{int } W, \text{pair}<\text{int}, \text{int}> arr[], \text{int } n)$

{ Sort (arr , $arr+n$, my cmp);

double $res = 0.0$;

for ($\text{int } i = 0$; $i < n$; $i++$)

{ if ($arr[i].\text{second} \leq W$)

{ $res += arr[i].\text{first}$;

$W = W - arr[i].\text{second}$;

}

else $res += arr[i].\text{first} * ((\text{double}) W / arr[i].\text{second})$;

break;

return res ;

bool mycmp ($\text{pair}<\text{int}, \text{int}> a$, $\text{pair}<\text{int}, \text{int}> b$)

{ double $r1 = (\text{double}) a.\text{first} / a.\text{second}$;

double $r2 = (\text{double}) b.\text{first} / b.\text{second}$;

return $r1 > r2$;

Job Sequencing Problem

- Rules:
- one unit by Every job.
 - Only one job can be assigned at a time.
 - Time Starts with 0.

Ip:

	J ₀	J ₁	J ₂	J ₃
deadline	4 ✓	1	1	1 ✓
profit	70	80	30	100

	J ₃	J ₀			
	0	1	2	3	4

Op: 170

Ip:

	J ₀	J ₁	J ₂	J ₃
deadline	2 ✓	2 ✓	3	3 ✓
profit	50	60	20	30

	J ₀	J ₁	J ₃
	0	1	2

Op: 140

Ip:

	J ₀	J ₁	J ₂	J ₃	J ₄
deadline	2	1	2	1	3
profit	100	50	70	20	30

Op: 180

	J ₁	J ₀	J ₄
	0	1	2

- Sort jobs in decreasing order of profit.
- Initialize the result as first job in the sorted list. Again Assign the latest possible slot.
- Do following for the remaining (n-1) jobs.
 - If this job cannot be added, ignore it.
 - Else add it to the latest possible slot.

	J ₀	J ₁	J ₂	J ₃	J ₄
deadline	2	1	2	1	3
profit	100	50	10	20	30

After Sorting

	J ₀	J ₁	J ₄	J ₃	J ₂
deadline	2	1	3	1	2
profit	100	50	30	20	10

	J ₄	J ₀	J ₄
	0	1	2

⇒

	J ₀	J ₁	J ₂	J ₃	J ₄
deadline	4	1	1	5	5
profit	50	5	20	10	80

After Sorting

	J ₄	J ₀	J ₂	J ₃	J ₁
deadline	5	4	1	5	1
profit	80	50	20	10	5

Op: 160

	J ₂	J ₃	J ₀	J ₄
	0	1	2	3

Huffman Coding (Introduction)

- used for Lossless Compression.
- Variable length Coding.

for compressed data we can get the original data by using decompression algorithms.

Example Problem:

* a b a a b a c a - - - - -

100 characters

8 x 100 = 800 bits
(or)
100 bytes

Frequencies

a - 70 00
b - 20 01
c - 10 10

Greedy Idea:

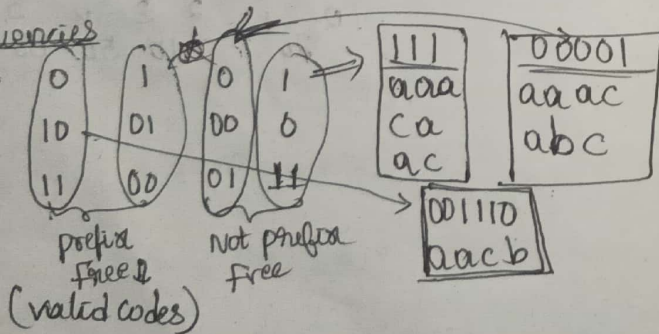
The most frequent character has smallest code

Prefix Requirement for Decompression:

No code should be prefix of any other

Frequencies

a - 70
b - 20
c - 10



$$70 \times 1 + 20 \times 2 + 10 \times 2 = 130 \text{ Bits}$$

for Fixed length Coding 200 Bits required

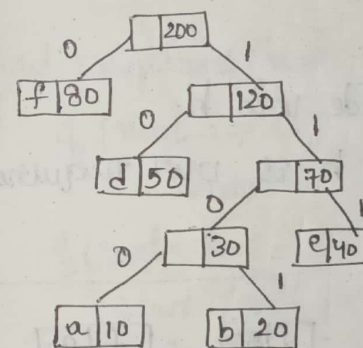
By Variable length Coding we save 70 bits

Huffman Algorithm (High level Idea)

Tip: ['a', 'd', 'b', 'e', 'f']

[10, 50, 20, 40, 80]

1) Build a Binary Tree



* Every input character is a leaf

* Every left child edge is labelled as 0 and right edge as 1.

* Every root to leaf path represents Huffman code of the leaf

2) Traverse the Binary Tree and print the codes

f 0
d 10
a 1100
b 1101
e 111

IP: ['a', 'd', 'b', 'e', 'f']
[10, 50, 20, 40, 80]

1) Create leaf nodes and build a Min Heap h of all the leaves initially.

2) while $h.size() > 1$:

a) left = $h.extractMin()$

b) right = $h.extractMin()$

c) create a new tree node with

⊛ character a \$

⊛ frequency as (left.freq + right.freq)

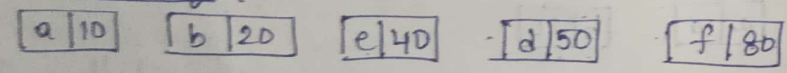
⊛ Left and right children as left and right respectively.

d) Insert the new node into h .

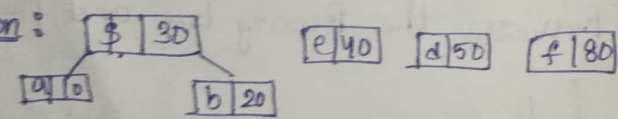
3) the only node left in h is not required

Binary Tree.

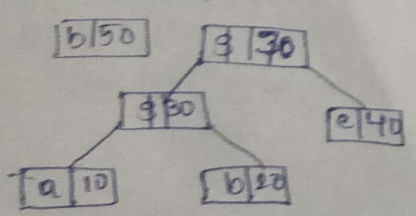
contents of h :



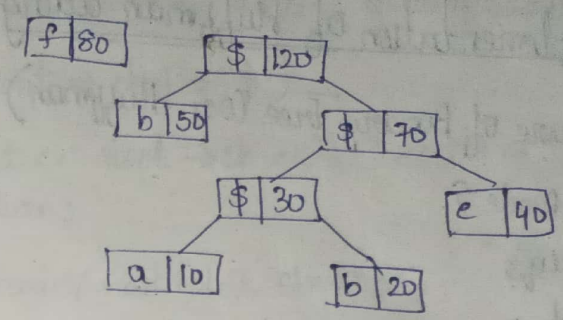
Ist Iteration:



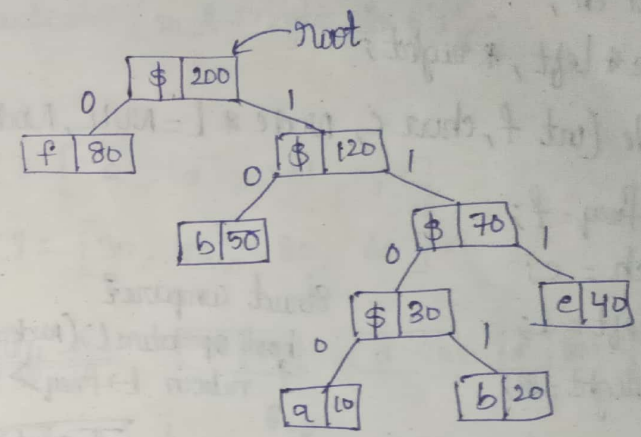
IInd Iteration:



III rd:



IV th:



Void printCodes (root, str = "")

if (root == NULL)
return

if (root.ch != \$)

print (root.ch + "" + str)
return

printCodes (root->left, str+"0");

printCodes (root->right, str+"1");

Printing Huffman codes from the Built Binary Tree

o/p: f 0
b 10
a 1100
b 1101
e 111

C++ Implementation of Huffman Coding:

Structure of Binary Tree (or Huffman) Tree Node

struct Node {

int freq;

char ch;

Node *left, *right;

Node (int f, char c, Node *L=NULL, Node *R=NULL)

{ freq = f;

ch = c;

left = L;

right = R;

};

struct compare {

bool operator () (Node *l, Node *r)

{ return l->freq > r->freq;

};

T.C = $O(n \log n)$

void printCodes (int arr[], int freq[], int n) { AS = $O(n)$ }

{ priority_queue < Node*, vector < Node* >, compare > h;

for (int i=0; i<n; i++)

h.push (new Node (arr[i], freq[i])); } $O(n \log n)$

while (h.size() > 1)

{ Node *L = h.top(); h.pop(); } $O(n \log n)$

Node *R = h.top(); h.pop();

Node *node = new Node ('\$', L->freq + R->freq, L, R);

h.push (node);

}

printCodes (h.top(), "");

}

void printCodes (Node *root, string str)

{ if (root->ch != '\$')

{ cout << root->ch << " " << str << "\n";

return;

printCodes (root->left, str + "0"); } $O(n)$

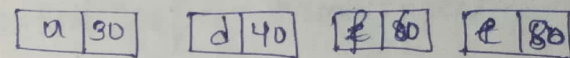
printCodes (root->right, str + "1");

}

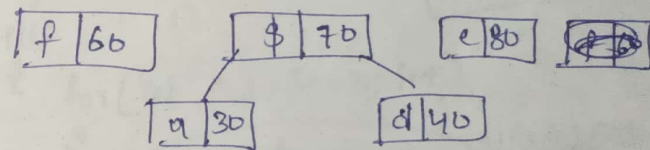
arr[] = {'a', 'd', 'e', 'f'}

freq[] = {30, 40, 80, 60}

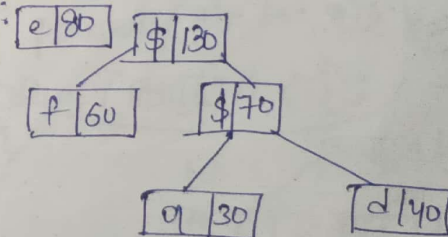
Initially h:



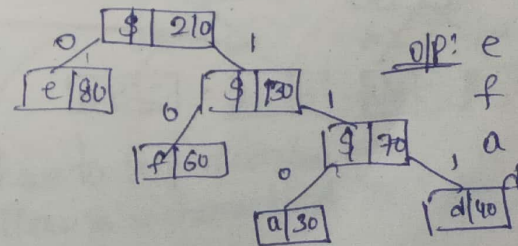
After 1st Iteration:



After 2nd:



after 3rd:



o/p: e 0
f 10
a 110
d 111